



Práctica Banco Nexus (Etapa 1): Bitácora

≡ Materia	Sistemas Distribuidos
≡ Etiquetas	actividad uabcs
🕒 Creado	May 12, 2026 3:57 PM



Ariel Felixmarco Perez Iglesias
Omar Armando Urquidez Castro
Jose Ernesto Guluarte Frias
Jose Carlos Victorio Coronado

Bitácora del Equipo

Banco Nexus

Bitácora del Equipo de Desarrollo

Práctica Distribuida - Etapa 1: Creación de la Base de Datos

1. Datos generales del proyecto

Banco Nexus es un proyecto académico que simula un sistema bancario con base de datos en MongoDB, backend en Node.js con Express y frontend en React con Vite. En la etapa actual del repositorio se implementan consultas de cuenta, visualización de saldo, historial de transacciones y operaciones básicas de depósito y retiro.

El objetivo de esta etapa es construir una base de datos funcional, inicializarla con datos simulados y conectarla con un backend que permita consultar la información desde una interfaz web.

2. Integrantes y roles

Integrante	Rol principal	Responsabilidades
Integrante 1 (Carlos Victorio)	Diseño de datos	Diseñar los documentos clientes , cuentas y transacciones ; definir relaciones; preparar datos simulados; crear el script de inicialización
Integrante 2 (Ariel Perez)	Backend	Crear la API REST con Express; preparar la conexión a MongoDB; implementar consultas de cuenta; validar acceso a colecciones; apoyar en la carga de datos
Integrante 3 (Ernesto Guluarte)	Frontend	Diseñar la primera interfaz; consultar una cuenta desde el backend; mostrar saldo, datos básicos y

Integrante	Rol principal	Responsabilidades
		movimientos
Integrante 4 (Omar Urquidez)	Infraestructura y documentación	Preparar instalación local de MongoDB; documentar conexión simple sin réplica; verificar consistencia de datos entre nodos; registrar problemas y soluciones

3. Objetivo de la etapa

Construir la base de datos `banco_nexus` en MongoDB con datos de prueba consistentes y conectarla a una aplicación funcional que permita:

- consultar una cuenta por número,
- mostrar datos del cliente y saldo actual,
- visualizar transacciones recientes e historial,
- probar la conexión entre MongoDB, backend y frontend.

4. Modelo de datos

La base de datos utiliza tres colecciones principales:

- `clientes`
- `cuentas`
- `transacciones`

4.1 Relación entre colecciones

- Un **cliente** puede tener una o más **cuentas**
- Cada **cuenta** pertenece a un **cliente**
- Cada **transacción** pertenece a una **cuenta**

Relación lógica:

`cliente -> cuenta -> transacción`

4.2 Colección `clientes`

Representa a los titulares de las cuentas bancarias.

Campo	Tipo	Descripción
<code>_id</code>	ObjectId	Identificador automático generado por MongoDB
<code>nombre</code>	String	Nombre completo del cliente
<code>curp</code>	String	Identificador único del cliente
<code>telefono</code>	String	Número telefónico de contacto
<code>email</code>	String	Correo electrónico

4.3 Colección `cuentas`

Representa las cuentas asociadas a los clientes.

Campo	Tipo	Descripción
<code>_id</code>	ObjectId	Identificador automático generado por MongoDB
<code>cuenta</code>	String	Número de cuenta
<code>cliente</code>	String	CURP del titular de la cuenta
<code>tipo</code>	String	Tipo de cuenta: <code>ahorro</code> , <code>corriente</code> o <code>nómina</code>
<code>saldo</code>	Number	Saldo actual
<code>fechaApertura</code>	Date	Fecha de apertura
<code>activa</code>	Boolean	Estado de la cuenta

4.4 Colección `transacciones`

Representa los movimientos realizados sobre una cuenta.

Campo	Tipo	Descripción
<code>_id</code>	ObjectId	Identificador automático generado por MongoDB
<code>cuenta</code>	String	Número de cuenta asociada
<code>tipo</code>	String	Tipo de movimiento: <code>deposito</code> o <code>retiro</code>
<code>monto</code>	Number	Cantidad operada
<code>saldoResultante</code>	Number	Saldo de la cuenta después de la operación
<code>fecha</code>	Date	Fecha y hora del movimiento
<code>descripcion</code>	String	Descripción del movimiento

5. Datos simulados cargados

El repositorio actual incluye un script de inicialización con datos fijos y reutilizables en `backend/seed.js`.

Contenido cargado por el script:

- 15 clientes simulados
- 15 cuentas bancarias
- múltiples transacciones de prueba distribuidas por cuenta

Los datos incluyen:

- nombres completos,
- CURP simuladas,
- correos electrónicos,
- teléfonos,
- números de cuenta,
- saldos,
- fechas de apertura,
- movimientos de depósito y retiro.

Esto permite probar la aplicación sin depender de capturas manuales de datos.

6. Evidencia del proyecto actual por integrante

Integrante 1 - Diseño de base de datos

Actividades realizadas:

- definición de las tres colecciones principales;
- modelado de la relación `cliente -> cuenta -> transacción` ;
- preparación de 15 clientes simulados con CURP, correo y teléfono;
- definición de cuentas asociadas y movimientos de ejemplo;
- estructura del script de inicialización con inserción de documentos.

Evidencia en el repositorio:

- `backend/models.js`
- `backend/seed.js`

Observaciones:

- El script actual utiliza datos determinísticos, lo cual favorece pruebas repetibles.
- Se crean índices para `curp` , `cuenta` , `cliente` , `cuenta` en transacciones y `fecha` .

Integrante 2 - Backend

Actividades realizadas:

- implementación del backend en Node.js con Express;
- creación del módulo de conexión reusable a MongoDB;
- implementación del endpoint principal `/api/cuenta/:cuenta` ;
- validación de acceso a las colecciones del proyecto;
- apoyo en el uso del script de carga de datos.

Evidencia en el repositorio:

- `backend/db.js`
- `backend/server.js`

Rutas implementadas en la versión actual:

- `GET /health`
- `GET /api/clientes`
- `GET /api/cuenta/:cuenta`
- `GET /api/historial/:cuenta`
- `POST /api/deposito`
- `POST /api/retiro`

Observaciones:

- La etapa 1 pedía principalmente la consulta de cuenta.

- El repositorio actual extiende esa funcionalidad con historial, depósitos y retiros, lo cual representa alcance adicional sobre la consigna base.

Integrante 3 - Frontend

Actividades realizadas:

- diseño de la primera interfaz de consulta de cuentas;
- formulario con campo de número de cuenta y botón de consulta;
- consumo del endpoint `/api/cuenta/:cuenta;`
- visualización de saldo, titular, tipo de cuenta y fecha de apertura;
- visualización de movimientos recientes;
- integración adicional con historial y formularios de operaciones.

Evidencia en el repositorio:

- `frontend/src/App.jsx`
- `frontend/src/api/bankApi.js`
- `frontend/src/components/AccountSearch.jsx`
- `frontend/src/components/SummaryCards.jsx`
- `frontend/src/components/TransactionsTable.jsx`

Alcance adicional implementado en la versión actual:

- gráfica de evolución del saldo;
- formularios de depósito y retiro;
- recarga de información después de una operación.

Integrante 4 - Infraestructura y documentación

Actividades realizadas:

- preparación del entorno local de MongoDB para ejecución individual;
- documentación del proceso de instalación y conexión local;
- definición de una conexión simple usando `mongodb://localhost:27017;`
- verificación de consistencia mediante conteos y consultas;
- registro de problemas frecuentes de conexión y ejecución.

Evidencia en el repositorio:

- `docs/Integrante4_Documentacion_MongoDB.md`
- `backend/db.js`
- `Makefile`

Observaciones:

- En esta etapa la conexión es local y simple, sin réplica configurada.

- El proyecto está preparado para que cada integrante pueda ejecutar la misma base de datos en su equipo.

7. Estructura actual del proyecto

```
banco_nexus/  
├── backend/  
│   ├── db.js  
│   ├── models.js  
│   ├── seed.js  
│   ├── server.js  
│   ├── package.json  
│   └── package-lock.json  
├── frontend/  
│   ├── index.html  
│   ├── package.json  
│   ├── vite.config.js  
│   └── src/  
│       ├── App.jsx  
│       ├── api/  
│       ├── components/  
│       ├── utils/  
│       └── styles.css  
├── docs/  
│   ├── Bitacora_Equipo_BancoNexus.md  
│   └── Integrante4_Documentacion_MongoDB.md  
└── Makefile
```

8. Componentes principales del proyecto actual

8.1 Script de inicialización

Archivo:

- `backend/seed.js`

Función:

- limpiar colecciones previas;
- insertar clientes, cuentas y transacciones;
- crear índices;
- dejar una base lista para pruebas.

8.2 Conexión a MongoDB

Archivo:

- `backend/db.js`

Características actuales:

- conexión centralizada;
- reutilizable desde el backend;
- nombre de base configurable por variable de entorno;
- cierre limpio del cliente al terminar el proceso.

8.3 API REST

Archivo:

- `backend/server.js`

Responsabilidad:

- exponer los datos de MongoDB a través de Express;
- consultar cuentas y movimientos;
- procesar depósitos y retiros;
- responder al frontend en formato JSON.

8.4 Frontend

Archivos principales:

- `frontend/src/App.jsx`
- `frontend/src/api/bankApi.js`

Responsabilidad:

- capturar el número de cuenta;
- consultar datos al backend;
- mostrar saldo, titular y transacciones;
- permitir pruebas funcionales desde una interfaz simple.

9. Instrucciones de ejecución

9.1 Requisitos

- Node.js 18 o superior
- MongoDB ejecutándose en `localhost:27017`

9.2 Inicializar la base de datos

Desde la raíz del proyecto:

```
make db-seed
```

O manualmente:

```
cd backend
node seed.js
```

9.3 Iniciar el backend

```
cd backend
npm install
```

```
npm start
```

9.4 Iniciar el frontend

```
cd frontend
npm install
npm run dev
```

9.5 Verificación básica

Backend:

```
curl http://localhost:3001/health
curl http://localhost:3001/api/cuenta/1000000001
```

Frontend:

- abrir la URL que muestre Vite en la terminal;
- capturar una cuenta de prueba;
- verificar saldo, titular y movimientos.

10. Bitácora resumida de trabajo

Actividad	Responsable	Estado
Diseño de colecciones y relaciones	Integrante 1	Completado
Preparación de datos simulados	Integrante 1	Completado
Script de inicialización de la base	Integrante 1	Completado
Módulo de conexión a MongoDB	Integrante 2	Completado
Endpoint de consulta de cuenta	Integrante 2	Completado
Validación de acceso a colecciones	Integrante 2	Completado
Interfaz de consulta de cuenta	Integrante 3	Completado
Visualización de saldo y movimientos	Integrante 3	Completado
Instalación local de MongoDB	Integrante 4	Completado
Verificación de consistencia de datos	Integrante 4	Completado
Documentación técnica de MongoDB	Integrante 4	Completado

11. Relación con la rúbrica

Diseño del modelo de datos

El proyecto cuenta con tres colecciones coherentes, relaciones definidas y datos suficientes para pruebas funcionales.

Implementación del script inicial

El repositorio actual incluye un script reutilizable que limpia, inserta y crea índices sobre la base de datos.

Conexión con MongoDB

La conexión está separada en un módulo propio y es reutilizada por el backend.

Trabajo colaborativo y documentación

La división por roles está clara y los entregables pueden asociarse con archivos concretos dentro del proyecto.

12. Observaciones finales

La versión actual del repositorio ya supera parcialmente la consigna mínima de la etapa 1, ya que además de la consulta de cuenta incorpora historial, depósitos, retiros y una interfaz más completa. Sin embargo, para fines de evaluación, este documento se centra en evidenciar que la base de datos, la carga inicial y la conexión con el backend sí corresponden al objetivo principal de la rúbrica.